

Papers of the
1990 MIT-ACM Undergraduate
Computer Science Conference

Papers of the
1990 MIT-ACM Undergraduate
Computer Science Conference

"Fault Tolerant Design for Midrange Routines: Let's Get It Done
works"

"MITeX: A Work in Progress"

William H. Jones

"A User Interface Server for the X Toolkit and the Motif
Widgets"

Barry J. Smith

"Before C4 Try GLASS"

Michael J. Smith

"The MIDAS System: A Machine-Independent Framework
for the Automatic Derivation and Application of
Machine Specific Optimizations"

Norm Lloyd

"An Ideal Future for Information Automation"

Michael B. Baker

"Switch Statement Optimization for 16-bit CPUs"

Ellen Sorensen

Conference chairman: Robert E. Finkel

Special thanks to MIT-ACM Undergraduate

Switch Statement Optimization for the 80x86

Ellen Spertus

1 Introduction

In recent years, C has become one of the most popular programming languages, providing efficiency not currently obtainable in many higher level languages and allowing greater abstraction and portability than assembly language. Generating efficient code from a high-level language is a difficult task for compiler writers but is absolutely crucial in allowing programmers to move up from assembly language. One feature of C that makes it easy to optimize is the small number of constructs that are part of the language itself: *if* statements, *for* and *while* loops, procedure linkage, and *switch* statements. Inherently, little optimization can be done on conditionals, but great possibilities exist for optimizing loops, procedure linkage, and switch statements. Loop optimization is a well-explored field, and several optimizations in procedure linkage (such as register calling conventions and inlining) are known, but switch statements have received relatively little study, despite their prominence in C, Pascal and Lisp.¹

The functionality of the switch statement can be shown by an example:

```
switch(num_people) {
    case 1:
        printf("Play solitaire.\n");
        break;
    case 2:
        printf("Play gin rummy.\n");
        break;
    case 3:
    case 6:
        printf("Play hearts.\n");
        break;
    case 4:
    case 5:
    case 7:
    case 8:
    case 9:
        printf("Play bridge.\n");2
        break;
    default:
        printf("Too many people for a card party.\n");
        break;
}
```

This segment of C code looks at a number stored in *num_people* and offers advice on what card game they should play. Note that different values (such as 3 and 6) can be mapped to the same code block. The default case is used when the argument is not equal to any of the listed numbers. Of course, code more substantial than printing usually appears in the case bodies. The above is meant only as a trivial example.

2 Optimizations

No easily accessible research exists on how large switch statements tend to be in actual C programs. In typical large projects, both short and long case statements can be found, with the size defined by the number of cases listed. Many code generation strategies are available, each producing better code in different circumstances. An optimizing compiler should have several different strategies for translating switch statements, particularly on an architecture as idiosyncratic as the 80x86 family (as opposed to more RISC-like processors). In a summer in Microsoft's Systems

¹In Pascal and Lisp, they are called *case* statements.

²With various schemes for rotating dummies, any set of 4, 5, 7, 8, or 9 people can satisfactorily play bridge.

Languages group, I was able to explore and implement different switch statement generation techniques.

In the discussion below, estimates of resource usage are only meant to be a first approximation, and code is sometimes slightly simplified. Readers unfamiliar with the 80x86 family should refer to the appendix for relevant information.

2.1 Linear Search

At first, the switch statement may appear as straight-forward and uncondusive to optimization as simple conditional statements; indeed, one could translate the above example into a conditional:

```
if (num_people == 1)
    printf("Play solitaire\n");
else if (num_people == 2)
    printf("Play gin rummy.\n");
else if (num_people == 3 || num_people == 6)
    printf("Play hearts.\n");
.
.
.
```

This sort of transformation is easy for the compiler writer to implement but generates sub-optimal code in many cases. If n different case values appear, the space used and execution time will both be $O(n)$. The above would translate to something like:

```
mov     ax, [bp+4]      ; Put num_people into the ax register

cmp     ax, 1           ; If it is equal to one,...
je      code_1_loc      ; ...branch to the first block

cmp     ax, 2
je      code_2_loc

cmp     ax, 3
je      code_3_6_loc

cmp     ax, 6
je      code_3_6_loc

.
.
.
```

As the size of the switch statement increases, the $O(n)$ time grows less acceptable. For example, 50×2 instructions would execute if the 50th value were the one matched. Although the compiler can possibly win by trusting the programmer to list the most common cases earliest to minimize the number of compares, the default selection must necessarily occur after every possible compare.

2.2 Binary Search

For larger case statements, binary search executes more quickly in the average case. Putting aside our original example, let us consider a switch statement that has different cases for all the numbers from 0 to n , where n is large, for example, 100. We could transform this into the following assembly code:

```

; Assume ax holds the variable

cmp     ax, 50                ; half of 100
jl      less_than_50         ; If less than 50
jg      more_than_50         ; If greater than 50
jmp     code_50_loc          ; If equal to 50

```

```

less_than_50:
    cmp     ax, 25
    .
    .
    .

```

This would execute in $O(\lg n)$ time, a significant improvement, although only for large n . Because the constant factor is approximately twice as large, binary search is only good when n is big. Also, like linear search, it uses $O(n)$ space, but with a larger constant factor. If the compiler is optimizing for code size instead of execution speed, linear search is always better than binary search.³

2.3 Jump Tables

For the most recent example, a strategy exists that executes in constant ($O(1)$) time. A “jump table” can be used, looking up the offset to jump to in a static array:

```

; If the value is not in range, jump to default location
cmp     ax, 100
jg      default_loc
cmp     ax, 0
jl      default_loc

move    table_base, bx      ; Put table base into base reg.
move    ax, si              ; Put the value into the index reg.
jmp     [bx+si]             ; Jump to value stored in the array

table_base:
    code_0_loc
    code_1_loc
    code_2_loc
    code_3_loc

```

If the switch variable is equal to 2, for example, the jump statement will go to the label specified at offset 2 from the base of the jump table. If any cases from the middle are “missing” and should go to the default case, the address of the default location can be put into the appropriate slots. Execution time is thus independent of the jump table size. However, the size may be worse than $O(n)$, if the case values are sparse. Consider the following switch statement to take logarithms base 10:

³The first use of binary search for switch statements that I have seen is from Microsoft C 4.0. Ralph Ryan and Dave Weil wrote the switch statement generation code.

```

switch(number) {
    case 1:
        return(0);
    case 10:
        return(1);
    case 100:
        return(2);
    case 1000:
        return(3);
    default:
        return(-1);          /* Illegal value */
}

```

If this were converted to a jump table, one thousand words of storage would be used, the vast majority containing the location of the default label. Determining our times in terms of n (here: 4) is thus insufficient. Let us use r to denote the range, which is here 1000. Jump tables thus use $O(1)$ time and $O(r)$ space. Only when the ratio r/n is sufficiently low should jump tables be used.

Intermezzo -- What we're ignoring

There are actually more such factors that we have swept under the rug, such as the distribution of the cases. In our dense example, switching on numbers from 0 to 100, a jump table is superior if the jumps are to diverse destinations. In contrast, if all the numbers from 0 to 50 go to one label, and all from 51 to 100 go to another, a modified linear search is ideal:

```

; Check if in first range
cmp    ax, 0
jl     code_default_loc
cmp    ax, 51
jl     code_0_through_50_loc

; Check if in second range
cmp    ax, 101
jl     code_51_through_100_loc

```

This code is both compact and speedy. Because distribution is difficult to quantize into a single variable, we will ignore it for now. An optimizing compiler, however, should consider distribution. To my knowledge, most C compilers do not or only do so to a small extent.

2.4 Scaling Jump Tables

In addition to cases like the one above, we want to use jump tables in cases where the lowest case arm is not zero. We would have to insert a subtract of this lowest value before indexing into the jump table, i.e. if the case values range from 80 to 100, a check should be inserting that the actual value is in this range, sending it to the default label otherwise, after which the base of 80 should be subtracted off, to avoid wasting space for 79 unneeded labels in the jump table. Additionally, dividing by a constant factor can be useful.⁴ For example, the case values in a switch statement might be too sparse for a jump table, but they might all be divisible by the same constant:

⁴This idea was suggested to me by Hans Spiller.

```

switch(decade)
{
    case 0:
        printf("The invention of the airplane.\n");
        break;
    case 10:
        printf("World War I.\n");
        break;
    case 20:
        printf("The stock market crash\n");
        break;
    case 30:
        printf("The Great Depression\n");
        break;
    .
    .
    .
    case 90:
        printf("Spertus gets Ph.D.\n");
        break;
}

```

If we divided our variable by 10, jumping straight to the default case if there was a remainder, we could use the quotient as an index into a jump table, generating a minimum amount of code. Of course, real life is not perfect, and the 80x86 family is often even worse. On the 8088, integer division can take over a 100 clock cycles. In comparison, an integer compare only takes about 3 cycles. We thus only want to do a divide if 1) the case statement is so large that the time penalty is absorbed, 2) we are compiling for a processor for which divide is relatively cheaper, 3) the user has chosen to optimize for space instead of speed, or 4) the division can be done with shifts instead of a divide.⁵ Microsoft C 6.0, forthcoming, for which I revamped the switch statement code, considers all the factors mentioned here and above.

2.5 Dual Tables

Another interesting method involves the 80x86 "string" scan instruction pair, *rep scas*, which scans through memory, stopping when it points to a value equal to the value in *ax*, storing its offset into the *cx* register. This may then be added to the base of another table to get the jump target. For example, if the actual value matches the third value in the first table, the *scas* instruction will store 3 into *cx*, which can be added to the base of the second table to get the correct label. The default case is handled by copying the actual value of the variable at run-time into the first table and having the location of the default code at the corresponding offset in the destination table. That way, whether the value is covered by one of the case arms or the default case, a match will be found. In practice, this is not a good strategy on the 80x86 family. For much the same reason as linear search, it uses $O(n)$ space and time. The constant factors, however, make it slower than linear search, in addition to its being unable to take advantage of ranges. Also, the strategy modifies its code (specifically, a data table stored in the code segment) at run-time, a questionable strategy, as it makes the code non-reentrant, a serious defect when virtual or physical parallel processors are being used.

⁵Shifting a number right by m divides it by 2^m . Astute readers might object that shifting right does not let you look at the quotient as division would allow you to; however, this can be simulated by anding the variable with an appropriate mask before shifting it down to determine what its remainder module 2^m would be.

3 Combinations of Strategies

The above discussions assumed that one strategy was preferable for any given switch statement. In practice, this is not the case. Consider a large switch statement that would be well suited for a jump table, except that it has half a dozen values not near the others. Microsoft C 6.0 would handle the dense cases with a jump table, and the half dozen other cases would be handled with linear search, binary search, or a jump table, or a combination thereof. Exploring all of these possibilities at compile-time can be time-consuming; fortunately, users can select whether the optimization phase should be run or not.

Another sort of combination arises from the ANSI requirement that switches can be done on long integers. Microsoft C represents integers as 16 bits and long integers as 32 bits. Because most 80x86 computers are 16-bit machines, the above scheme must be modified. For example, to see if a long integer variable is equal to 16, the following code would be generated:

```

mov     ax, [low_word]
mov     dx, [high_word]

; Must check both words
cmp     ax, 16
jne     not_16
cmp     dx, 0
je      code_16_loc

; Check next possibility

```

A linear search on long integers would thus take approximately twice as much space and time as on 16-bit integers. Fortunately, optimizations are available. For example, suppose all of the non-default values have the same upper word (i.e. are all between $a \cdot 2^{16}$ and $(a+1) \cdot 2^{16}$, for some constant a). We could then first ensure that the high word was correct and then check each of the lower words. Instead of doing $2 \cdot n$ jump/comparison pairs, we do only $n+1$, a substantial savings. Code size would similarly be reduced. Some professional C compilers on the market at the time of my research (and probably to this day) failed to use this optimization and generated this sort of code:

```

; Assume dx:ax holds the long integer value

; Is it equal to $10000?
cmp     dx, 1
jne     not_first
cmp     ax, 0
je      code_10000_loc

not_first:
; Is it equal to $10005?
cmp     dx, 1
jne     not_second
cmp     ax, 5
je      code_10005_loc

.
.
.

not_(n-1)th:
; Is it equal to $100F3?
cmp     dx, 1
jne     code_default_loc
cmp     ax, 0f3h
jne     code_default_loc

```

It would be much more efficient to just check `dx` at the beginning and never re-check it. Similarly, if all the low words were the same, `ax` could be checked for once and for all at the start. Alternately, in the above example, \$10000 could be subtracted off at the start, with the case values scaled accordingly. The subtraction method is especially good when all of the legal values are within 2^{16} (65536) of each other but do not have the same upper word; for example: 8, 11, 30, 57, 91, 108, 65536, and 65537. This could be handled by subtracting the lowest value, 8, from `dx:ax`, after which it could be checked that the value has 0 in `dx`, after which only `ax` would need to be checked to determine the destination.

Of course, in some switch statements, both the high and low words will be widely distributed. This could be handled by any of the above strategies or their legal combinations. For example, a binary search can be done on the values of the high words, after which a linear search, binary search, or jump table (or any combinations thereof) could be done for each of the possible low words.

4 Conclusion

Although switch statements are used in C, Pascal, and Lisp, little research has been published on how to optimize them. Several strategies are known by scattered individuals, but little formal documentation exists on the subject. I searched the literature before making my changes to the Microsoft C compiler and was unable to find any references to binary search for switch statements or the idea of combining different methods for a single switch statement. Much more literature exists, for example, on loop optimization. Although loop optimization is more important, switch statement optimization is definitely worth doing. The classic "dragon book" on compiler design contains nothing on switch statement optimization despite the existence of all the opportunities described above.

5 Acknowledgments

I would like to thank Mark Roberts, my manager at Microsoft during the summer of 1988, when I did this work. I would also like to thank my friend Nate Osgood for letting me bounce ideas off of him.